

Iterative Statements

Introduction to Programming

StayAhead Training; Sep, 2024

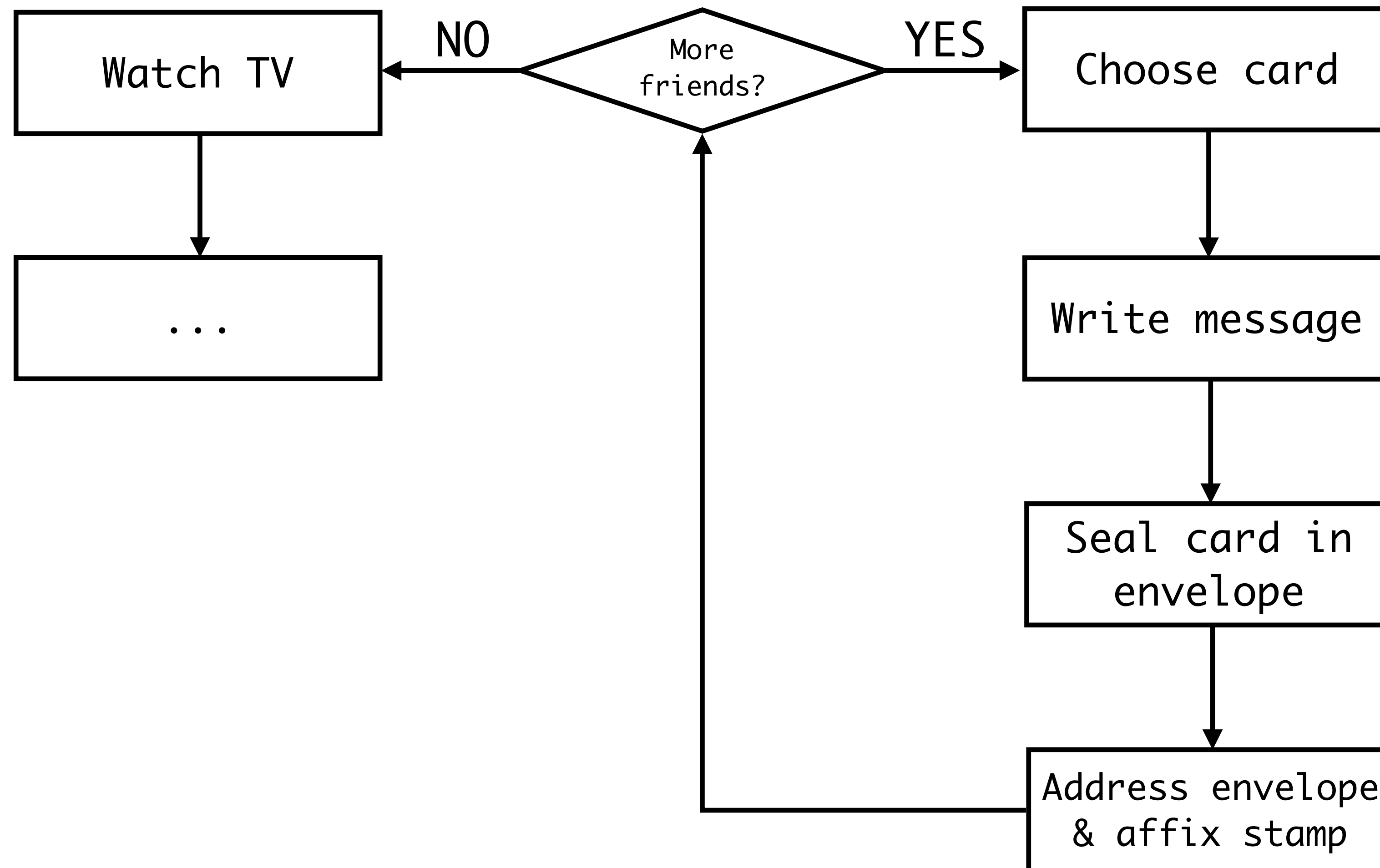
Introduction

Iterative Statements

- **Iterative statement:** one in which the assoc. block is executed n times
- Recall that a block is a group of statements that may or may not be executed

Introduction

Iterative Statements



While Loop

Iterative Statements

- Pseudocode, e.g.:
 - SET Count to 1
 - WHILE Count $\leq$ 3
 - OUTPUT Count to stdout
 - SET Count to Count + 1
 - ENDWHILE

While Loop

Iterative Statements

- `x = 1`
- `while x < 6:`
- `print(x)`
- `x += 1`

For Loop

Iterative Statements

- The traditional for loop is shorthand for a while loop
- Each comprises three parts:
 1. Control variable initialisation (before first iteration)
 2. Boolean expression (at the start of each iteration)
 3. Control variable reassignment (at the end of each iteration)
- For loops are, arguably, harder to reason about than are while loops

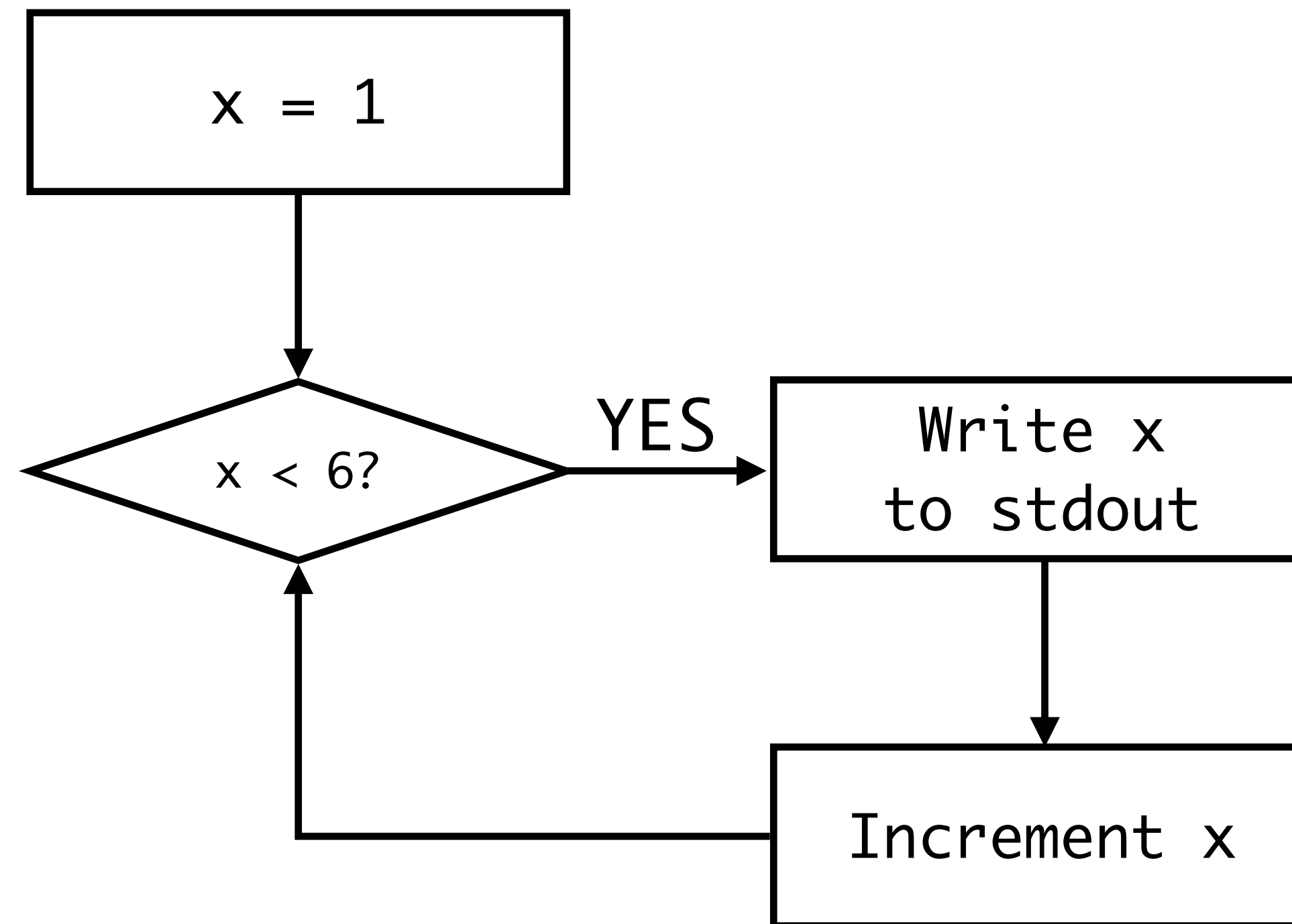
For Loop

Iterative Statements

- `for x in range(1, 6):`
- `print(x)`

For Loop

Iterative Statements



For Loop

Iterative Statement

Iteration #	Value of x at start	x < 6?	Value of x at end
1	1	True	2
2	2	True	3
3	3	True	4
4	4	True	5
5	5	True	6
6	6	False	

Code Blocks (again)

Iterative Statements

- `for x in range(1, 6):`
- `# instruction to be repeated code block`
- `print(x)`

Traversing an Array

Iterative Statements

- Pseudocode, e.g.:
 - FOREACH Name in Names
 - OUTPUT Name to stdout
 - ENDFOREACH

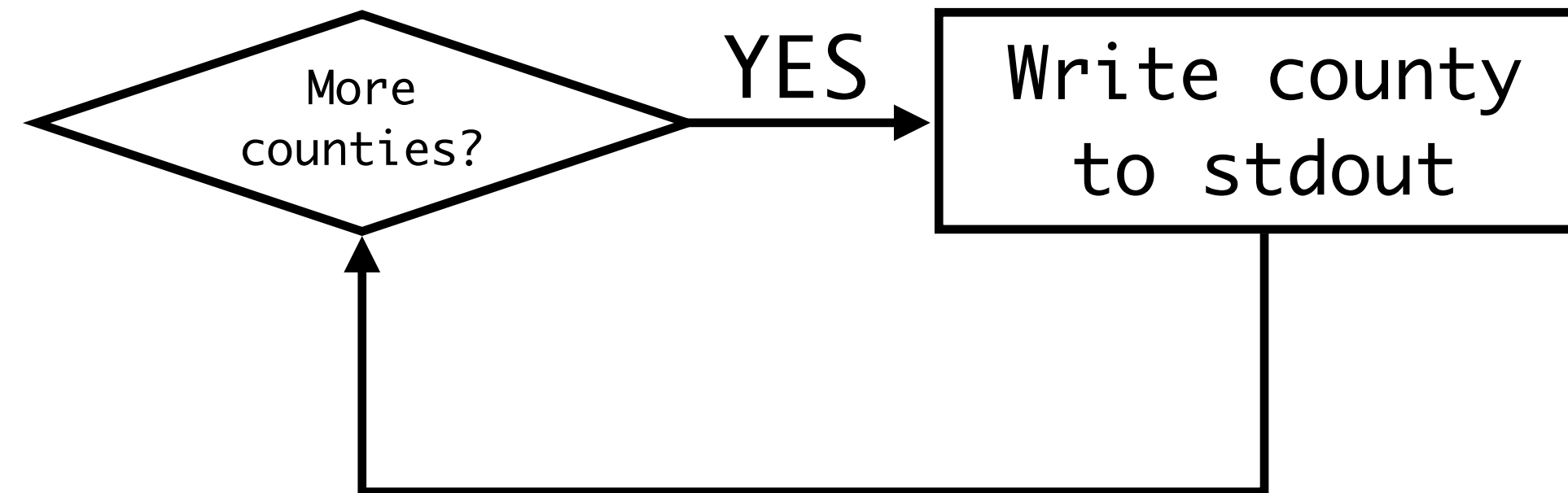
Traversing an Array

Iterative Statements

- `for` county `in` counties:
- `print(county)`

Traversing an Array

Iterative Statements



Break

Iterative Statements

- `x = 1`
- `while True:`
- `if x == 6:`
- `break`
- `print(x)`
- `x += 1`

Continue

Iterative Statements

- `x = 1`
- `while x < 6:`
- `if x == 3:`
- `x += 1`
- `continue`
- `print(x)`
- `x += 1`